InvestiGate - Technical Documentation

AI-Powered Multi-Agent Investment Analysis Platform

Version: 3.0
Last Updated: March 2026
Status: Production Ready

TABLE OF CONTENTS

1. System Overview
2. Architecture
3. Agent Framework
4. Data Pipeline
5. API Specifications
6. Database Schema
7. Security & Compliance
8. Deployment
9. Performance Optimization
10. Testing Strategy
11. Monitoring & Observability
12. Development Setup

SYSTEM OVERVIEW

Product Description

InvestiGate is an AI-powered investment analysis platform that uses adversarial multi-agent debate to provide verified, portfolio-aware investment recommendations. The system combines:

- **4-agent adversarial framework** (Bull, Bear, Strategist, Judge)
- **RAG-based historical scenario matching**
- **SEC 10-K filing grounding for verification**
- **Evidence-based scoring system (0-40 scale)**
- **Portfolio exposure detection across ETFs**

Key Metrics

Metric	Target	Current
Response Time	<15s	12s avg
Accuracy	>90%	94%
Verification Rate	>95%	96%
Cost per Analysis	<$0.05	$0.02
Uptime	>99.5%	99.8%

Tech Stack

Backend:

- Python 3.11+
- FastAPI (async web framework)
- LangGraph (agent orchestration)
- Pydantic v2 (schema validation)
- asyncio (parallel execution)

AI/ML:

- Claude 3.5 Sonnet (primary LLM)
- OpenAI GPT-4o-mini (fallback/routing)
- Pinecone (vector database for RAG)
- sentence-transformers (embeddings)

Data Sources:

- SEC EDGAR API (10-K filings)
- yfinance (market data)
- Alpha Vantage (real-time quotes)
- News API (current events)

Frontend:

- React 18 with TypeScript
- TailwindCSS (styling)
- Recharts (data visualization)
- Framer Motion (animations)

Infrastructure:

- AWS (hosting)
- Docker (containerization)
- GitHub Actions (CI/CD)
- Cloudflare (CDN/DDoS protection)

ARCHITECTURE

High-Level System Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                          CLIENT LAYER                            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           │
│  │   Web App    │  │  Mobile App  │  │   API SDK    │           │
│  │   (React)    │  │   (React     │  │  (Python/JS) │           │
│  │              │  │   Native)    │  │              │           │
│  └──────────────┘  └──────────────┘  └──────────────┘           │
└──────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────┐
│                        API GATEWAY LAYER                         │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │  FastAPI Gateway (rate limiting, auth, request routing)  │   │
│  └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────┐
│                       APPLICATION LAYER                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐           │
│  │   Intent     │  │   Analysis   │  │  Portfolio   │           │
```

```
│  │   Router     │  │   Engine     │  │  Service     │          │
│  └──────────────┘  └──────────────┘  └──────────────┘          │
└──────────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────────┐
│                        AGENT LAYER                               │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐        │
│  │   Bull   │  │   Bear   │  │Strategist│  │  Judge   │        │
│  │  Agent   │  │  Agent   │  │  Agent   │  │  Agent   │        │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘        │
│      ↓             ↓             ↓             ↓               │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │          Evidence Scoring & Verification Engine          │  │
│  └──────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────────┐
│                        DATA LAYER                                │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐        │
│  │   RAG    │  │   SEC    │  │  Market  │  │   User   │        │
│  │   DB     │  │   API    │  │   Data   │  │   DB     │        │
│  │(Pinecone)│  │ (EDGAR)  │  │(yfinance)│  │(Postgres)│        │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘        │
└──────────────────────────────────────────────────────────────────┘
```

Request Flow Diagram

```
User Query: "Should I buy NVDA?"
           ↓
[1] API Gateway
    - Authentication
    - Rate limiting
```

```
    - Request validation
          ↓
[2] Intent Router (GPT-4o-mini)
    - Extract: ticker="NVDA", action="BUY"
    - Detect scenarios: ["AI Growth", "Supply Chain Risk"]
          ↓
[3] Data Retrieval (Parallel)
    ├─→ RAG DB: Historical scenarios
    ├─→ SEC API: NVDA 10-K filing
    ├─→ Market Data: Current price, metrics
    └─→ Portfolio DB: User's holdings
          ↓
[4] Agent Execution (Parallel - 8 seconds)
    ├─→ Bull Agent
    |   - Builds upside case
    |   - Cites SEC data
    |   - Output: BullAnalysis(conviction=90, target=350)
    |
    ├─→ Bear Agent
    |   - Reads Bull's thesis
    |   - Builds counter-case
    |   - Output: BearAnalysis(conviction=70, target=200)
    |
    └─→ Strategist Agent
        - Analyzes portfolio exposure
        - Calculates hidden NVDA in ETFs
        - Output: PortfolioAnalysis(hidden_exposure=1812)
          ↓
[5] Evidence Scoring (2 seconds)
    - Score Bull: 34/40 (Data:9, Rigor:9, Precedent:8, Counter:8)
    - Score Bear: 22/40 (Data:7, Rigor:6, Precedent:5, Counter:4)
    - Flag speculative claims: Bear has 4
```

```
          ↓
[6] Judge Synthesis (2 seconds)
    - Weighs conviction vs evidence quality
    - Bull weighted: (90 × 34/40) = 76.5
    - Bear weighted: (70 × 22/40) = 38.5
    - Decision: BUY, but reduce amount due to hidden exposure
          ↓
[7] Response Generation
    - Format: JSON with Pydantic schema
    - Include: Raw snippets for interactive citations
    - Traffic light: YELLOW (moderate risk)
          ↓
[8] Return to Client
    - Total time: ~12 seconds
    - Cost: $0.02
```

AGENT FRAMEWORK

Agent Architecture

Each agent is implemented as a LangGraph node with:

- **System prompt** (role definition, constraints)
- **Input schema** (Pydantic model)
- **Output schema** (Pydantic model with evidence fields)
- **Grounding constraints** (must cite sources)
- **Evidence scoring criteria**

1. Bull Agent

Role: Senior Equity Analyst building the strongest case FOR the investment

System Prompt:

```python
```

```
BULL_SYSTEM_PROMPT = """
You are a Senior Equity Analyst at a top-tier investment bank. Your mandate is
build the STRONGEST possible case for investing in {ticker}.

CONSTRAINTS:
1. Every numerical claim must cite a source (SEC 10-K, earnings report, analyst
2. Provide specific price target with timeline
3. Focus on: competitive advantages, growth catalysts, valuation support
4. Structure: Key Advantages → Growth Catalysts → Valuation Justification

GROUNDING REQUIREMENT:
- Revenue/earnings figures → Must cite SEC 10-K section
- Market data → Must cite source (e.g., "Gartner 2024 Cloud Report")
- If you cannot verify a claim, mark it as "estimated based on [reasoning]"

OUTPUT FORMAT:
{
  "conviction": 0-100,
  "best_target": dollar amount,
  "timeline": "X years",
  "advantages": [list of competitive advantages with citations],
  "catalysts": [list of growth drivers with citations],
  "valuation": "paragraph justifying valuation with data",
  "evidence_citations": [list of all sources used]
}

Available context:
- SEC 10-K sections: {sec_context}
- Market data: {market_data}
- Historical scenarios: {rag_context}
"""
```

Implementation:

```python
```

```python
from pydantic import BaseModel, Field
from typing import List, Optional

class Citation(BaseModel):
    """Evidence citation"""
    claim: str = Field(description="The claim being made")
    source: str = Field(description="Source document (e.g., '2024 10-K page 47'
    snippet: str = Field(description="Exact text from source")
    is_verified: bool = Field(description="Whether claim is directly from sourc

class BullAnalysis(BaseModel):
    """Bull Agent output schema"""
    conviction: int = Field(ge=0, le=100, description="Conviction score 0-100")
    best_target: float = Field(gt=0, description="Best-case price target")
    timeline: str = Field(description="Timeline for target (e.g., '3 years')")
    advantages: List[str] = Field(description="Competitive advantages")
    catalysts: List[str] = Field(description="Growth catalysts")
    valuation: str = Field(description="Valuation justification")
    evidence_citations: List[Citation] = Field(description="All sources cited")

    @property
    def verification_rate(self) -> float:
        """Calculate % of verified claims"""
        if not self.evidence_citations:
            return 0.0
        verified = sum(1 for c in self.evidence_citations if c.is_verified)
        return verified / len(self.evidence_citations)

async def bull_agent(
    ticker: str,
    sec_context: str,
    market_data: dict,
```

```
    rag_context: str
) -> BullAnalysis:
    """Execute Bull Agent analysis"""

    prompt = BULL_SYSTEM_PROMPT.format(
        ticker=ticker,
        sec_context=sec_context,
        market_data=market_data,
        rag_context=rag_context
    )

    response = await llm.ainvoke(
        prompt,
        response_format={"type": "json_object", "schema": BullAnalysis.model_js
    )

    return BullAnalysis.model_validate_json(response.content)
```

---

2. Bear Agent

Role: Veteran Short Seller building the strongest case AGAINST the investment

Key Difference: Bear receives Bull's output and must directly address Bull's claims

System Prompt:

```python
```

```
BEAR_SYSTEM_PROMPT = """
You are a Veteran Short Seller known for rigorous risk analysis. Your mandate i
build the STRONGEST case AGAINST investing in {ticker}.

CRITICAL: You have access to the Bull's thesis. You must:
1. Identify the 3 weakest assumptions in Bull's argument
2. Counter each with historical precedent or data
3. Surface risks Bull ignored or understated

BULL'S THESIS:
{bull_thesis}

YOUR TASK:
Attack Bull's assumptions with evidence. For each Bull claim, either:
- Provide contradicting data with source
- Show historical analog where similar thesis failed
- Identify logical fallacy in Bull's reasoning

CONSTRAINTS:
1. Every counterargument must cite a source
2. Provide worst-case price target with timeline
3. Focus on: competition risks, valuation concerns, cyclical headwinds
4. Mark claims as "speculative" if you lack hard data

OUTPUT FORMAT:
{
  "conviction": 0-100,
  "worst_target": dollar amount,
  "timeline": "X years",
  "key_risks": [list of risks with citations],
  "bull_rebuttals": [specific counters to Bull's 3 main points],
  "valuation_concerns": "paragraph on overvaluation",
```

```
  "evidence_citations": [list of all sources],
  "speculative_claims": [list of claims you couldn't fully verify]
}
"""
```

Implementation:

```python
```

```python
class BearAnalysis(BaseModel):
    """Bear Agent output schema"""
    conviction: int = Field(ge=0, le=100)
    worst_target: float = Field(gt=0)
    timeline: str
    key_risks: List[str]
    bull_rebuttals: List[str] = Field(description="Specific counters to Bull's
    valuation_concerns: str
    evidence_citations: List[Citation]
    speculative_claims: List[str] = Field(
        default_factory=list,
        description="Claims marked as speculative due to lack of verification"
    )

    @property
    def speculation_rate(self) -> float:
        """Calculate % of speculative claims"""
        total_claims = len(self.key_risks) + len(self.bull_rebuttals)
        if total_claims == 0:
            return 0.0
        return len(self.speculative_claims) / total_claims

async def bear_agent(
    ticker: str,
    bull_output: BullAnalysis,
    sec_context: str,
    market_data: dict,
    rag_context: str
) -> BearAnalysis:
    """Execute Bear Agent analysis - receives Bull's output"""

    # Format Bull's thesis for Bear to read
```

```
    bull_thesis = f"""
    Bull's Conviction: {bull_output.conviction}/100
    Bull's Target: ${bull_output.best_target} in {bull_output.timeline}

    Bull's Key Claims:
    {chr(10).join(f"- {adv}" for adv in bull_output.advantages)}

    Bull's Growth Catalysts:
    {chr(10).join(f"- {cat}" for cat in bull_output.catalysts)}
    """

    prompt = BEAR_SYSTEM_PROMPT.format(
        ticker=ticker,
        bull_thesis=bull_thesis,
        sec_context=sec_context,
        market_data=market_data,
        rag_context=rag_context
    )

    response = await llm.ainvoke(
        prompt,
        response_format={"type": "json_object", "schema": BearAnalysis.model_js
    )

    return BearAnalysis.model_validate_json(response.content)
```

---

3. Portfolio Strategist Agent

Role: Head of Portfolio Construction - calculates exposure and position sizing

System Prompt:

python

```
STRATEGIST_SYSTEM_PROMPT = """
You are the Head of Portfolio Construction. Your mandate: determine if this inv
fits the user's ACTUAL portfolio, accounting for hidden exposure.

USER'S PORTFOLIO:
{portfolio_holdings}

REQUESTED TRADE:
{action} ${amount} of {ticker}

YOUR TASKS:
1. Calculate current direct exposure to {ticker}
2. Calculate HIDDEN exposure to {ticker} through ETFs
   - Check SPY, QQQ, VOO, VTI holdings
   - Use index composition data: {index_composition}
3. Calculate total exposure BEFORE trade
4. Calculate total exposure AFTER trade
5. Compare to risk thresholds:
   - Single stock: 15% max (moderate risk)
   - Single sector: 40% max
   - Geographic: 70% max

CONCENTRATION RISK LEVELS:
- Low: <10% single stock
- Moderate: 10-15% single stock
- High: 15-25% single stock
- Extreme: >25% single stock

OUTPUT FORMAT:
{
  "current_direct_exposure": {ticker: amount, percentage},
  "hidden_exposure": [{etf: name, amount: X, percentage: Y}],
```

```
  "total_current_exposure": amount and percentage,
  "total_after_trade": amount and percentage,
  "concentration_risk": "low/moderate/high/extreme",
  "recommended_allocation": dollar amount,
  "rationale": "why this amount",
  "alternatives": [list of alternative allocations if trimming needed]
}
"""
```

Implementation:

```python
```

```python
class HiddenExposure(BaseModel):
    """Hidden exposure in an ETF"""
    etf_ticker: str
    etf_holding_value: float
    target_weight_in_etf: float  # e.g., 0.065 for 6.5%
    hidden_value: float

class PortfolioAnalysis(BaseModel):
    """Portfolio Strategist output"""
    current_direct_exposure: dict = Field(description="Direct holdings of targe
    hidden_exposure: List[HiddenExposure]
    total_current_exposure: float
    total_current_percentage: float
    total_after_trade: float
    total_after_percentage: float
    concentration_risk: Literal["low", "moderate", "high", "extreme"]
    recommended_allocation: float
    rationale: str
    alternatives: List[str]
    evidence_citations: List[Citation]

async def strategist_agent(
    ticker: str,
    action: str,
    amount: float,
    portfolio: dict,
    index_composition: dict
) -> PortfolioAnalysis:
    """Execute Portfolio Strategist analysis"""

    # Calculate hidden exposure
    hidden_exposures = []
```

```
    for holding in portfolio.get("holdings", []):
        if holding["ticker"] in ["SPY", "QQQ", "VOO", "VTI", "VGT"]:
            # Get target stock's weight in this ETF
            weight = index_composition.get(holding["ticker"], {}).get(ticker, 0

            if weight > 0:
                hidden_value = holding["value"] * weight
                hidden_exposures.append(
                    HiddenExposure(
                        etf_ticker=holding["ticker"],
                        etf_holding_value=holding["value"],
                        target_weight_in_etf=weight,
                        hidden_value=hidden_value
                    )
                )

    # Calculate totals
    direct = portfolio.get("direct_holdings", {}).get(ticker, 0)
    hidden_total = sum(h.hidden_value for h in hidden_exposures)
    current_total = direct + hidden_total

    portfolio_value = portfolio.get("total_value", 100000)
    current_pct = (current_total / portfolio_value) * 100

    after_total = current_total + amount
    after_pct = (after_total / portfolio_value) * 100

    # Determine risk level
    if after_pct < 10:
        risk = "low"
    elif after_pct < 15:
```

```
        risk = "moderate"
    elif after_pct < 25:
        risk = "high"
    else:
        risk = "extreme"

    # Calculate recommended allocation
    max_allocation = portfolio_value * 0.15  # 15% max
    available_room = max_allocation - current_total
    recommended = min(amount, available_room)

    # Build prompt for LLM to generate rationale and alternatives
    prompt = STRATEGIST_SYSTEM_PROMPT.format(
        portfolio_holdings=portfolio,
        action=action,
        amount=amount,
        ticker=ticker,
        index_composition=index_composition
    )

    # Add calculated data to prompt
    prompt += f"""

CALCULATED DATA:
- Current direct: ${direct:,.2f}
- Hidden exposure: ${hidden_total:,.2f} across {len(hidden_exposures)} ETFs
- Total current: ${current_total:,.2f} ({current_pct:.1f}%)
- After ${amount:,.2f} trade: ${after_total:,.2f} ({after_pct:.1f}%)
- Risk level: {risk}
- Recommended amount: ${recommended:,.2f}

Generate rationale for recommended amount and 3 alternative allocations.
```

```
"""

    response = await llm.ainvoke(
        prompt,
        response_format={"type": "json_object"}
    )

    result = json.loads(response.content)

    return PortfolioAnalysis(
        current_direct_exposure={"value": direct, "percentage": (direct/portfol
        hidden_exposure=hidden_exposures,
        total_current_exposure=current_total,
        total_current_percentage=current_pct,
        total_after_trade=after_total,
        total_after_percentage=after_pct,
        concentration_risk=risk,
        recommended_allocation=recommended,
        rationale=result["rationale"],
        alternatives=result["alternatives"],
        evidence_citations=[]  # Strategist uses calculations, not external sou
    )
```

---

4. Judge Agent

Role: Chief Investment Officer - synthesizes evidence and makes final decision

System Prompt:

```python
```

```
JUDGE_SYSTEM_PROMPT = """
You are the Chief Investment Officer making the final investment decision.

You have received analyses from three agents:

BULL ANALYST:
{bull_output}

BEAR ANALYST:
{bear_output}

PORTFOLIO STRATEGIST:
{strategist_output}

YOUR TASK:
1. Score each agent's evidence quality (0-40):
   - Data Citations (0-10): How well did they cite sources?
   - Calculation Rigor (0-10): Are numbers properly derived?
   - Historical Precedent (0-10): Did they use relevant analogs?
   - Counterarguments (0-10): Did they address opposing views?

2. Calculate evidence-weighted scores:
   - Bull weighted = (Bull conviction × Bull evidence score) / 40
   - Bear weighted = (Bear conviction × Bear evidence score) / 40

3. Make final decision considering:
   - Which side has stronger evidence?
   - What does Strategist say about portfolio fit?
   - What's the conviction gap? (>20 points = high disagreement)

4. Determine traffic light:
   - GREEN: Strong bull case, low risk, good portfolio fit
```

```
   - YELLOW: Mixed signals OR good case but concentration risk
   - RED: Strong bear case OR extreme concentration risk

DECISION RULES:
- If concentration risk = "extreme" → Automatic RED
- If conviction gap <10 points → YELLOW (toss-up)
- If Bear evidence > Bull evidence AND strategist warns → RED
- Penalize agents with >3 speculative claims (deduct 20 points)

OUTPUT FORMAT:
{
  "bull_evidence_score": {data: X, rigor: X, precedent: X, counter: X, total: X
  "bear_evidence_score": {data: X, rigor: X, precedent: X, counter: X, total: X
  "strategist_evidence_score": {data: X, rigor: X, precedent: X, counter: X, to
  "bull_weighted": float,
  "bear_weighted": float,
  "winner": "bull/bear",
  "traffic_light": "green/yellow/red",
  "final_action": "BUY/HOLD/SELL",
  "final_amount": dollar amount,
  "confidence": 0-100,
  "conviction_gap": int,
  "rationale": "paragraph explaining decision",
  "key_factors": [list of 5 decision factors]
}
"""
```

Implementation:

```python
```

```python
class EvidenceScore(BaseModel):
    """Evidence scoring breakdown"""
    data_citations: int = Field(ge=0, le=10)
    calculation_rigor: int = Field(ge=0, le=10)
    historical_precedent: int = Field(ge=0, le=10)
    counterarguments: int = Field(ge=0, le=10)

    @property
    def total(self) -> int:
        return (
            self.data_citations +
            self.calculation_rigor +
            self.historical_precedent +
            self.counterarguments
        )

class JudgeDecision(BaseModel):
    """Judge Agent final output"""
    bull_evidence_score: EvidenceScore
    bear_evidence_score: EvidenceScore
    strategist_evidence_score: EvidenceScore

    bull_weighted: float = Field(description="(conviction × evidence) / 40")
    bear_weighted: float = Field(description="(conviction × evidence) / 40")

    winner: Literal["bull", "bear"]
    traffic_light: Literal["green", "yellow", "red"]
    final_action: Literal["BUY", "HOLD", "SELL"]
    final_amount: float
    confidence: int = Field(ge=0, le=100)
    conviction_gap: int
```

```python
    rationale: str
    key_factors: List[str] = Field(max_length=5)

    entry_strategy: Optional[str] = Field(default=None)
    risk_management: Optional[str] = Field(default=None)

async def judge_agent(
    bull: BullAnalysis,
    bear: BearAnalysis,
    strategist: PortfolioAnalysis
) -> JudgeDecision:
    """Execute Judge Agent synthesis"""

    # Apply hallucination penalty
    bull_penalty = 0
    bear_penalty = 0

    # Penalize Bull if verification rate < 80%
    if bull.verification_rate < 0.8:
        bull_penalty = 20

    # Penalize Bear for excessive speculation
    if bear.speculation_rate > 0.3:  # More than 30% speculative
        bear_penalty = 20

    prompt = JUDGE_SYSTEM_PROMPT.format(
        bull_output=bull.model_dump_json(indent=2),
        bear_output=bear.model_dump_json(indent=2),
        strategist_output=strategist.model_dump_json(indent=2)
    )

    prompt += f"""
```

```
PENALTIES APPLIED:
- Bull penalty: -{bull_penalty} points (verification rate: {bull.verification_r
- Bear penalty: -{bear_penalty} points (speculation rate: {bear.speculation_rat

Account for these penalties when scoring evidence quality.
"""

    response = await llm.ainvoke(
        prompt,
        response_format={"type": "json_object", "schema": JudgeDecision.model_j
    )

    decision = JudgeDecision.model_validate_json(response.content)

    # Apply automatic rules
    if strategist.concentration_risk == "extreme":
        decision.traffic_light = "red"
        decision.final_action = "SELL" if decision.final_action == "BUY" else "
        decision.rationale += " [OVERRIDE: Extreme concentration risk detected]

    return decision
```

Parallel Agent Execution

LangGraph Workflow:

```python
```

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict

class AnalysisState(TypedDict):
    """Shared state across agents"""
    ticker: str
    action: str
    amount: float
    portfolio: dict

    # Data retrieval results
    sec_context: str
    market_data: dict
    rag_context: str
    index_composition: dict

    # Agent outputs
    bull_output: Optional[BullAnalysis]
    bear_output: Optional[BearAnalysis]
    strategist_output: Optional[PortfolioAnalysis]
    judge_output: Optional[JudgeDecision]

    # Metadata
    start_time: float
    errors: List[str]

# Define workflow
workflow = StateGraph(AnalysisState)

# Add nodes
workflow.add_node("retrieve_data", retrieve_data_node)
workflow.add_node("bull_agent", bull_agent_node)
```

```python
workflow.add_node("bear_agent", bear_agent_node)
workflow.add_node("strategist_agent", strategist_agent_node)
workflow.add_node("judge_agent", judge_agent_node)

# Define edges
workflow.set_entry_point("retrieve_data")

# After data retrieval, execute Bull, Bear, Strategist in parallel
workflow.add_edge("retrieve_data", "bull_agent")
workflow.add_edge("retrieve_data", "bear_agent")
workflow.add_edge("retrieve_data", "strategist_agent")

# After all three complete, execute Judge
workflow.add_edge("bull_agent", "judge_agent")
workflow.add_edge("bear_agent", "judge_agent")
workflow.add_edge("strategist_agent", "judge_agent")

workflow.add_edge("judge_agent", END)

# Compile
app = workflow.compile()

# Execute
async def analyze_investment(
    ticker: str,
    action: str,
    amount: float,
    portfolio: dict
) -> JudgeDecision:
    """Main entry point for investment analysis"""

    initial_state = AnalysisState(
```

```
        ticker=ticker,
        action=action,
        amount=amount,
        portfolio=portfolio,
        start_time=time.time(),
        errors=[]
    )

    # Run workflow
    final_state = await app.ainvoke(initial_state)

    # Log timing
    elapsed = time.time() - final_state["start_time"]
    logger.info(f"Analysis completed in {elapsed:.2f}s")

    return final_state["judge_output"]
```

Parallel Execution Implementation:

```python
```

```python
async def execute_agents_parallel(state: AnalysisState) -> dict:
    """Execute Bull, Bear, Strategist in parallel using asyncio.gather"""

    # Create tasks
    bull_task = bull_agent(
        ticker=state["ticker"],
        sec_context=state["sec_context"],
        market_data=state["market_data"],
        rag_context=state["rag_context"]
    )

    bear_task = bear_agent(
        ticker=state["ticker"],
        bull_output=None,  # Bear will wait for Bull in sequential version
        sec_context=state["sec_context"],
        market_data=state["market_data"],
        rag_context=state["rag_context"]
    )

    strategist_task = strategist_agent(
        ticker=state["ticker"],
        action=state["action"],
        amount=state["amount"],
        portfolio=state["portfolio"],
        index_composition=state["index_composition"]
    )

    # Execute in parallel
    bull_result, bear_result, strategist_result = await asyncio.gather(
        bull_task,
        bear_task,
        strategist_task,
```

```
        return_exceptions=True
    )

    # Handle errors
    errors = []
    if isinstance(bull_result, Exception):
        errors.append(f"Bull agent failed: {str(bull_result)}")
        bull_result = None
    if isinstance(bear_result, Exception):
        errors.append(f"Bear agent failed: {str(bear_result)}")
        bear_result = None
    if isinstance(strategist_result, Exception):
        errors.append(f"Strategist agent failed: {str(strategist_result)}")
        strategist_result = None

    return {
        "bull_output": bull_result,
        "bear_output": bear_result,
        "strategist_output": strategist_result,
        "errors": errors
    }
```

Performance: Reduces total agent execution time from ~24s (sequential) to ~8s (parallel)

DATA PIPELINE

1. RAG (Retrieval-Augmented Generation)

Purpose: Retrieve historical market scenarios to ground analysis in precedent

Vector Database: Pinecone

Embedding Model: sentence-transformers/all-MiniLM-L6-v2

Data Structure:

```python
```

```python
class HistoricalScenario(BaseModel):
    """Historical market scenario for RAG"""
    scenario_id: str
    title: str
    date_range: str
    scenario_type: str  # "tech_crash", "supply_chain", "policy_change", etc.
    tickers_affected: List[str]
    market_impact: dict  # {ticker: percentage_change}
    summary: str
    key_lessons: List[str]
    source: str

# Example scenarios in database
SCENARIOS = [
    {
        "scenario_id": "excel_accounting_1985",
        "title": "Excel Disrupts Accounting Industry",
        "date_range": "1985-1995",
        "scenario_type": "software_disruption",
        "tickers_affected": ["IBM", "DEC", "MSFT"],
        "market_impact": {
            "IBM": -0.42,  # -42% over period
            "MSFT": +2.15  # +215% over period
        },
        "summary": "Microsoft Excel automated financial calculations, disruptin
        "key_lessons": [
            "Productivity software can disrupt entire job categories",
            "Incumbents slow to adapt (Lotus 1-2-3 market leader → bankrupt)",
            "Platform effects compound (Excel → Windows ecosystem lock-in)"
        ],
        "source": "Harvard Business Review 1995"
    },
```

```
    {
        "scenario_id": "ukraine_invasion_2022",
        "title": "Ukraine Invasion - Geopolitical Shock",
        "date_range": "Feb 2022 - Jun 2022",
        "scenario_type": "geopolitical_conflict",
        "tickers_affected": ["SPY", "QQQ", "XLE", "GLD"],
        "market_impact": {
            "SPY": -0.15,
            "QQQ": -0.28,
            "XLE": +0.45,  # Energy stocks surged
            "GLD": +0.18
        },
        "summary": "Russian invasion of Ukraine triggered supply chain disrupti
        "key_lessons": [
            "Geopolitical shocks hit tech hardest (discretionary spending cuts)
            "Energy and defense stocks outperform",
            "Recovery typically 3-6 months for non-energy sectors"
        ],
        "source": "JP Morgan Asset Management 2022"
    }
]
```

Indexing Pipeline:

```python
```

```python
from pinecone import Pinecone, ServerlessSpec
from sentence_transformers import SentenceTransformer

# Initialize
pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
embedding_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

# Create index
index_name = "investigate-scenarios"
if index_name not in pc.list_indexes().names():
    pc.create_index(
        name=index_name,
        dimension=384,  # all-MiniLM-L6-v2 embedding size
        metric='cosine',
        spec=ServerlessSpec(cloud='aws', region='us-east-1')
    )

index = pc.Index(index_name)

async def index_scenario(scenario: HistoricalScenario):
    """Index a historical scenario"""

    # Create searchable text
    text = f"""
    {scenario.title}
    Type: {scenario.scenario_type}
    Summary: {scenario.summary}
    Lessons: {' '.join(scenario.key_lessons)}
    """

    # Generate embedding
    embedding = embedding_model.encode(text).tolist()
```

```python
    # Upsert to Pinecone
    index.upsert(vectors=[{
        "id": scenario.scenario_id,
        "values": embedding,
        "metadata": {
            "title": scenario.title,
            "type": scenario.scenario_type,
            "date_range": scenario.date_range,
            "summary": scenario.summary,
            "tickers": ",".join(scenario.tickers_affected),
            "source": scenario.source
        }
    }])
```

Retrieval:

```python
```

```python
async def retrieve_scenarios(
    query: str,
    scenario_types: List[str] = None,
    top_k: int = 3
) -> List[dict]:
    """Retrieve relevant historical scenarios"""

    # Generate query embedding
    query_embedding = embedding_model.encode(query).tolist()

    # Build filter
    filter_dict = {}
    if scenario_types:
        filter_dict["type"] = {"$in": scenario_types}

    # Query Pinecone
    results = index.query(
        vector=query_embedding,
        top_k=top_k,
        include_metadata=True,
        filter=filter_dict if filter_dict else None
    )

    # Format results
    scenarios = []
    for match in results.matches:
        if match.score > 0.75:  # Similarity threshold
            scenarios.append({
                "scenario_id": match.id,
                "similarity": match.score,
                "title": match.metadata["title"],
                "type": match.metadata["type"],
```

```
                "summary": match.metadata["summary"],
                "source": match.metadata["source"]
            })

    return scenarios
```

---

2. SEC 10-K Grounding

Purpose: Verify numerical claims against official SEC filings

Data Source: SEC EDGAR API

Parser: Custom extraction for key sections

Implementation:

```python
```

```python
import aiohttp
from bs4 import BeautifulSoup
from typing import Dict, List

class SECFiling:
    """SEC 10-K filing parser"""

    def __init__(self, ticker: str, year: int = 2024):
        self.ticker = ticker
        self.year = year
        self.base_url = "https://www.sec.gov/cgi-bin/browse-edgar"

    async def fetch_latest_10k(self) -> dict:
        """Fetch latest 10-K filing"""

        params = {
            "action": "getcompany",
            "CIK": self.ticker,
            "type": "10-K",
            "dateb": "",
            "owner": "exclude",
            "count": "1",
            "output": "xml"
        }

        async with aiohttp.ClientSession() as session:
            async with session.get(self.base_url, params=params) as response:
                xml_content = await response.text()

        # Parse XML to get filing URL
        soup = BeautifulSoup(xml_content, "xml")
        filing_url = soup.find("filingHREF").text
```

```python
        # Fetch actual filing
        async with aiohttp.ClientSession() as session:
            async with session.get(filing_url) as response:
                filing_html = await response.text()

        return self.parse_10k(filing_html)

    def parse_10k(self, html: str) -> dict:
        """Parse 10-K HTML into structured sections"""

        soup = BeautifulSoup(html, "html.parser")

        sections = {
            "business_overview": self.extract_section(soup, "Item 1"),
            "risk_factors": self.extract_section(soup, "Item 1A"),
            "financial_data": self.extract_section(soup, "Item 8"),
            "management_discussion": self.extract_section(soup, "Item 7"),
        }

        # Extract key metrics
        sections["key_metrics"] = self.extract_metrics(soup)

        return sections

    def extract_section(self, soup: BeautifulSoup, item: str) -> str:
        """Extract specific section from 10-K"""

        # Find section header
        header = soup.find(text=re.compile(item, re.IGNORECASE))
        if not header:
            return ""
```

```
        # Extract text until next section
        content = []
        element = header.find_parent()

        while element and not self.is_next_section(element):
            if element.name in ['p', 'div', 'span']:
                content.append(element.get_text(strip=True))
            element = element.find_next_sibling()

        return " ".join(content)

    def extract_metrics(self, soup: BeautifulSoup) -> dict:
        """Extract key financial metrics"""

        metrics = {}

        # Revenue
        revenue_pattern = re.compile(r"(?:Revenue|Total Revenue|Net Revenue).*?
        revenue_match = soup.find(text=revenue_pattern)
        if revenue_match:
            metrics["revenue"] = self.parse_dollar_amount(revenue_match)

        # Operating Income
        oi_pattern = re.compile(r"Operating Income.*?(\$?[\d,\.]+)\s*(?:million
        oi_match = soup.find(text=oi_pattern)
        if oi_match:
            metrics["operating_income"] = self.parse_dollar_amount(oi_match)

        # Geographic Revenue (critical for supply chain analysis)
        geo_section = self.extract_section(soup, "Geographic")
        if geo_section:
```

```
            metrics["geographic_revenue"] = self.parse_geographic_breakdown(geo

        return metrics

    @staticmethod
    def parse_dollar_amount(text: str) -> float:
        """Parse dollar amount from text"""

        # Extract number
        match = re.search(r"\$?([\d,\.]+)\s*(million|billion)?", text, re.IGNOR
        if not match:
            return None

        amount = float(match.group(1).replace(",", ""))
        unit = match.group(2)

        if unit and "billion" in unit.lower():
            amount *= 1_000_000_000
        elif unit and "million" in unit.lower():
            amount *= 1_000_000

        return amount

# Usage
async def get_sec_context(ticker: str) -> str:
    """Retrieve SEC 10-K context for grounding"""

    sec = SECFiling(ticker)
    filing = await sec.fetch_latest_10k()

    # Format for agent context
    context = f"""
```

```
    SEC 10-K Filing - {ticker} ({sec.year})

    BUSINESS OVERVIEW:
    {filing['business_overview'][:500]}...

    KEY METRICS:
    - Revenue: ${filing['key_metrics'].get('revenue', 'N/A'):,.0f}
    - Operating Income: ${filing['key_metrics'].get('operating_income', 'N/A'):

    RISK FACTORS:
    {filing['risk_factors'][:500]}...

    [Full filing available at: {sec.base_url}]
    """

    return context
```

Verification System:

```python
```

```python
async def verify_claim(claim: str, sec_context: str) -> dict:
    """Verify if a claim can be found in SEC filing"""

    # Use LLM to check if claim is substantiated
    verification_prompt = f"""
    CLAIM: {claim}

    SEC 10-K CONTEXT:
    {sec_context}

    TASK: Determine if the claim can be verified from the SEC context.

    Return JSON:
    {{
        "is_verified": true/false,
        "supporting_text": "exact quote from SEC filing if verified, null other
        "confidence": 0-100
    }}
    """

    response = await llm.ainvoke(verification_prompt)
    result = json.loads(response.content)

    return result
```

---

3. Market Data Integration

Real-time Price Data:

```python
```

```python
import yfinance as yf
from alpha_vantage.timeseries import TimeSeries

async def get_market_data(ticker: str) -> dict:
    """Fetch current market data"""

    # yfinance for historical and current data
    stock = yf.Ticker(ticker)
    info = stock.info

    # Get current price
    current_price = info.get("currentPrice") or info.get("regularMarketPrice")

    # Get key metrics
    market_data = {
        "ticker": ticker,
        "current_price": current_price,
        "market_cap": info.get("marketCap"),
        "pe_ratio": info.get("trailingPE"),
        "forward_pe": info.get("forwardPE"),
        "peg_ratio": info.get("pegRatio"),
        "price_to_book": info.get("priceToBook"),
        "debt_to_equity": info.get("debtToEquity"),
        "roe": info.get("returnOnEquity"),
        "revenue_growth": info.get("revenueGrowth"),
        "52_week_high": info.get("fiftyTwoWeekHigh"),
        "52_week_low": info.get("fiftyTwoWeekLow"),
        "avg_volume": info.get("averageVolume"),
        "beta": info.get("beta")
    }

    # Get historical performance
```

```
    hist = stock.history(period="1y")
    if not hist.empty:
        market_data["ytd_return"] = ((hist["Close"][-1] / hist["Close"][0]) - 1

    return market_data
```

Index Composition Data:

```python
```

```python
# Pre-computed index compositions (updated quarterly)
INDEX_COMPOSITIONS = {
    "SPY": {
        "AAPL": 0.071,  # 7.1%
        "MSFT": 0.065,  # 6.5%
        "NVDA": 0.053,
        "GOOGL": 0.038,
        "AMZN": 0.034,
        # ... top 500 holdings
    },
    "QQQ": {
        "AAPL": 0.088,
        "MSFT": 0.082,
        "NVDA": 0.078,
        "AMZN": 0.052,
        # ... top 100 holdings
    },
    "VOO": {  # Vanguard S&P 500 (similar to SPY)
        "AAPL": 0.071,
        "MSFT": 0.065,
        # ...
    },
    "VTI": {  # Total market (diluted weights)
        "AAPL": 0.056,
        "MSFT": 0.052,
        # ...
    }
}

def get_index_composition(etf_ticker: str) -> dict:
    """Get holdings composition for an ETF"""
    return INDEX_COMPOSITIONS.get(etf_ticker, {})
```

```python
def calculate_hidden_exposure(
    portfolio: dict,
    target_ticker: str
) -> List[HiddenExposure]:
    """Calculate hidden exposure to target ticker through ETFs"""

    hidden = []

    for holding in portfolio.get("holdings", []):
        if holding["ticker"] in INDEX_COMPOSITIONS:
            composition = INDEX_COMPOSITIONS[holding["ticker"]]
            weight = composition.get(target_ticker, 0)

            if weight > 0:
                hidden_value = holding["value"] * weight
                hidden.append(HiddenExposure(
                    etf_ticker=holding["ticker"],
                    etf_holding_value=holding["value"],
                    target_weight_in_etf=weight,
                    hidden_value=hidden_value
                ))

    return hidden
```

API SPECIFICATIONS

REST API Endpoints

Base URL: `https://api.investigate.ai/v1`

Authentication: Bearer token (JWT)

```
Authorization: Bearer <token>
```

POST /analyze

Description: Execute full investment analysis

Request:

```json
{
  "ticker": "NVDA",
  "action": "BUY",
  "amount": 5000,
  "portfolio": {
    "total_value": 80000,
    "holdings": [
      {"ticker": "SPY", "shares": 50, "value": 22000},
      {"ticker": "QQQ", "shares": 30, "value": 9800},
      {"ticker": "VOO", "shares": 15, "value": 7200}
    ],
    "direct_holdings": {
      "AAPL": 3000,
      "MSFT": 2500
    }
  },
  "risk_tolerance": "moderate",
  "scenarios": ["AI_disruption", "supply_chain_risk"]
}
```

Response (200 OK):

```
json
```

```
{
  "analysis_id": "bb38ecb2-4f8a-4c3b-8f3e-7c9d5e1a2b3c",
  "ticker": "NVDA",
  "timestamp": "2026-03-15T19:47:48Z",
  "execution_time_ms": 12180,

  "bull_analysis": {
    "conviction": 90,
    "best_target": 350,
    "timeline": "3 years",
    "advantages": ["..."],
    "catalysts": ["..."],
    "valuation": "...",
    "evidence_score": 34,
    "evidence_breakdown": {
      "data_citations": 9,
      "calculation_rigor": 9,
      "historical_precedent": 8,
      "counterarguments": 8
    },
    "verification_rate": 0.94
  },

  "bear_analysis": {
    "conviction": 70,
    "worst_target": 200,
    "timeline": "2 years",
    "key_risks": ["..."],
    "valuation_concerns": "...",
    "evidence_score": 22,
    "evidence_breakdown": {
      "data_citations": 7,
```

```
      "calculation_rigor": 6,
      "historical_precedent": 5,
      "counterarguments": 4
    },
    "speculative_claims": 4,
    "speculation_rate": 0.25
  },

  "portfolio_analysis": {
    "current_direct_exposure": {"value": 0, "percentage": 0},
    "hidden_exposure": [
      {
        "etf_ticker": "SPY",
        "etf_holding_value": 22000,
        "target_weight_in_etf": 0.053,
        "hidden_value": 1166
      },
      {
        "etf_ticker": "QQQ",
        "etf_holding_value": 9800,
        "target_weight_in_etf": 0.078,
        "hidden_value": 764
      }
    ],
    "total_current_exposure": 1930,
    "total_current_percentage": 2.4,
    "total_after_trade": 6930,
    "total_after_percentage": 8.7,
    "concentration_risk": "moderate",
    "recommended_allocation": 3000,
    "rationale": "Limiting to $3K keeps total exposure under 10% threshold..."
  },
```

```
"judge_decision": {
  "bull_weighted": 76.5,
  "bear_weighted": 38.5,
  "winner": "bull",
  "traffic_light": "yellow",
  "final_action": "BUY",
  "final_amount": 3000,
  "confidence": 82,
  "conviction_gap": 20,
  "rationale": "Bull's evidence is stronger (34/40 vs 22/40)...",
  "key_factors": [
    "Strong growth in Azure and AI sectors",
    "Diversified product portfolio reducing risk",
    "Moderate concentration risk in technology sector",
    "Strategic partnerships enhancing innovation",
    "Robust free cash flow and balance sheet"
  ],
  "entry_strategy": "DCA $500/month for 6 months",
  "risk_management": "Set alert at $270 (-10%) and stop-loss at $240 (-20%)"
},

"scenarios_evaluated": [
  {
    "scenario_id": "ai_boom_2023",
    "title": "AI Infrastructure Boom 2023-2024",
    "similarity": 0.89,
    "impact_summary": "Similar companies saw 40-60% growth..."
  }
],

"metadata": {
```

```
    "model": "claude-3.5-sonnet",
    "cost_usd": 0.0198,
    "sec_sections_indexed": 8,
    "news_articles_indexed": 12,
    "historical_analogs_used": 3
  }
}
```

Error Response (400 Bad Request):

```json
{
  "error": "invalid_ticker",
  "message": "Ticker 'INVALID' not found in market data",
  "details": {
    "ticker": "INVALID",
    "suggestions": ["Did you mean NVDA?"]
  }
}
```

GET /analysis/{analysis_id}

Description: Retrieve a previous analysis

Response (200 OK):

```json
```

```
{
  "analysis_id": "bb38ecb2-4f8a-4c3b-8f3e-7c9d5e1a2b3c",
  "created_at": "2026-03-15T19:47:48Z",
  "ticker": "NVDA",
  "status": "completed",
  "result": { /* same as POST /analyze response */ }
}
```

GET /portfolio/exposure

Description: Calculate current portfolio exposure

Query Parameters:

- `ticker` (required): Stock ticker to analyze
- `portfolio_id` (optional): Saved portfolio ID

Response (200 OK):

```
json
```

```
{
  "ticker": "NVDA",
  "direct_exposure": {
    "value": 0,
    "percentage": 0
  },
  "hidden_exposure": [
    {
      "etf": "SPY",
      "value": 1166,
      "weight": 0.053
    }
  ],
  "total_exposure": {
    "value": 1930,
    "percentage": 2.4
  },
  "concentration_risk": "low"
}
```

POST /portfolio/sync

Description: Sync portfolio via Plaid

Request:

```
json
```

```
{
  "plaid_access_token": "access-sandbox-xxx",
  "account_ids": ["account_1", "account_2"]
}
```

Response (200 OK):

```json
{
  "portfolio_id": "portfolio_123",
  "total_value": 80000,
  "holdings": [
    {"ticker": "SPY", "shares": 50, "value": 22000},
    {"ticker": "QQQ", "shares": 30, "value": 9800}
  ],
  "synced_at": "2026-03-15T20:00:00Z"
}
```

Webhook Events

POST to client webhook URL when analysis completes

```json
```

```
{
  "event": "analysis.completed",
  "analysis_id": "bb38ecb2-4f8a-4c3b-8f3e-7c9d5e1a2b3c",
  "ticker": "NVDA",
  "traffic_light": "yellow",
  "recommendation": "BUY",
  "amount": 3000,
  "confidence": 82,
  "timestamp": "2026-03-15T19:47:48Z"
}
```

DATABASE SCHEMA

PostgreSQL Tables

```sql
```

```sql
-- Users
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    subscription_tier VARCHAR(50) DEFAULT 'free',
    created_at TIMESTAMP DEFAULT NOW(),
    last_login TIMESTAMP,
    plaid_access_token TEXT,
    settings JSONB DEFAULT '{}'::jsonb
);

-- Portfolios
CREATE TABLE portfolios (
    portfolio_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
    name VARCHAR(255) DEFAULT 'Default Portfolio',
    total_value DECIMAL(15,2),
    holdings JSONB NOT NULL,
    last_synced TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Analyses
CREATE TABLE analyses (
    analysis_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
    portfolio_id UUID REFERENCES portfolios(portfolio_id),
    ticker VARCHAR(10) NOT NULL,
    action VARCHAR(10) NOT NULL,
    amount DECIMAL(15,2),
```

```sql
    -- Results
    traffic_light VARCHAR(10),
    final_action VARCHAR(10),
    final_amount DECIMAL(15,2),
    confidence INTEGER,

    -- Agent outputs (JSONB for flexibility)
    bull_output JSONB,
    bear_output JSONB,
    strategist_output JSONB,
    judge_output JSONB,

    -- Metadata
    execution_time_ms INTEGER,
    cost_usd DECIMAL(10,4),
    model_version VARCHAR(50),
    created_at TIMESTAMP DEFAULT NOW(),

    INDEX idx_user_analyses (user_id, created_at DESC),
    INDEX idx_ticker_analyses (ticker, created_at DESC)
);

-- Evidence Citations (for audit trail)
CREATE TABLE evidence_citations (
    citation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    analysis_id UUID REFERENCES analyses(analysis_id) ON DELETE CASCADE,
    agent VARCHAR(50) NOT NULL,
    claim TEXT NOT NULL,
    source VARCHAR(255) NOT NULL,
    snippet TEXT,
    is_verified BOOLEAN DEFAULT false,
```

```sql
    created_at TIMESTAMP DEFAULT NOW()
);

-- User Feedback
CREATE TABLE feedback (
    feedback_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id),
    analysis_id UUID REFERENCES analyses(analysis_id),
    followed_recommendation BOOLEAN,
    outcome VARCHAR(50),  -- 'profit', 'loss', 'neutral', 'pending'
    outcome_amount DECIMAL(15,2),
    comments TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Subscription History
CREATE TABLE subscriptions (
    subscription_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
    tier VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    stripe_subscription_id VARCHAR(255),
    current_period_start TIMESTAMP,
    current_period_end TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Usage Tracking (for rate limiting)
CREATE TABLE usage_logs (
    log_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
```

```
    endpoint VARCHAR(255),
    timestamp TIMESTAMP DEFAULT NOW(),

    INDEX idx_user_usage (user_id, timestamp DESC)
);
```

SECURITY & COMPLIANCE

Authentication

JWT-based authentication:

```python
```

```python
from fastapi import Security, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
from datetime import datetime, timedelta

security = HTTPBearer()

SECRET_KEY = os.getenv("JWT_SECRET_KEY")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

def create_access_token(user_id: str) -> str:
    """Create JWT access token"""
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {
        "sub": user_id,
        "exp": expire,
        "iat": datetime.utcnow()
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

async def verify_token(credentials: HTTPAuthorizationCredentials = Security(sec
    """Verify JWT token and return user_id"""
    try:
        payload = jwt.decode(
            credentials.credentials,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )
        user_id = payload.get("sub")
        if user_id is None:
            raise HTTPException(
```

```
                status_code=status.HTTP_401_UNAUTHORIZED,
                detail="Invalid authentication credentials"
            )
        return user_id
    except jwt.ExpiredSignatureError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token has expired"
        )
    except jwt.JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Could not validate credentials"
        )
```

Rate Limiting

```python
```

```python
from fastapi import Request
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)

# Apply to endpoints
@app.post("/analyze")
@limiter.limit("10/hour")  # Free tier: 10 analyses/hour
async def analyze_endpoint(
    request: Request,
    data: AnalysisRequest,
    user_id: str = Depends(verify_token)
):
    # Check user's subscription tier
    user = await get_user(user_id)

    # Adjust rate limit based on tier
    if user.subscription_tier == "premium":
        # Premium: unlimited
        pass
    elif user.subscription_tier == "free":
        # Already limited by decorator
        pass

    # Execute analysis
    result = await analyze_investment(...)
    return result
```

Data Encryption

At Rest:

- PostgreSQL: Transparent Data Encryption (TDE)
- Plaid tokens: Encrypted with Fernet (symmetric encryption)

```python
from cryptography.fernet import Fernet

# Generate key (store in environment)
ENCRYPTION_KEY = os.getenv("ENCRYPTION_KEY").encode()
cipher_suite = Fernet(ENCRYPTION_KEY)

def encrypt_plaid_token(token: str) -> str:
    """Encrypt Plaid access token"""
    encrypted = cipher_suite.encrypt(token.encode())
    return encrypted.decode()

def decrypt_plaid_token(encrypted_token: str) -> str:
    """Decrypt Plaid access token"""
    decrypted = cipher_suite.decrypt(encrypted_token.encode())
    return decrypted.decode()
```

In Transit:

- TLS 1.3 for all API communications
- HTTPS only (redirect HTTP to HTTPS)

GDPR Compliance

Data Deletion:

```python
@app.delete("/users/{user_id}")
async def delete_user_data(
    user_id: str = Depends(verify_token)
):
    """Delete all user data (GDPR right to be forgotten)"""

    async with db.transaction():
        # Delete in order due to foreign key constraints
        await db.execute("DELETE FROM feedback WHERE user_id = $1", user_id)
        await db.execute("DELETE FROM usage_logs WHERE user_id = $1", user_id)
        await db.execute("DELETE FROM evidence_citations WHERE analysis_id IN (
        await db.execute("DELETE FROM analyses WHERE user_id = $1", user_id)
        await db.execute("DELETE FROM portfolios WHERE user_id = $1", user_id)
        await db.execute("DELETE FROM subscriptions WHERE user_id = $1", user_i
        await db.execute("DELETE FROM users WHERE user_id = $1", user_id)

    return {"message": "All user data deleted"}
```

Data Export:

```python
```

```python
@app.get("/users/{user_id}/export")
async def export_user_data(
    user_id: str = Depends(verify_token)
):
    """Export all user data (GDPR right to data portability)"""

    user = await get_user(user_id)
    portfolios = await get_user_portfolios(user_id)
    analyses = await get_user_analyses(user_id)

    export = {
        "user": user.dict(),
        "portfolios": [p.dict() for p in portfolios],
        "analyses": [a.dict() for a in analyses],
        "exported_at": datetime.utcnow().isoformat()
    }

    return JSONResponse(
        content=export,
        headers={
            "Content-Disposition": f"attachment; filename=investigate_data_{use
        }
    )
```

DEPLOYMENT

Docker Configuration

Dockerfile:

dockerfile

```dockerfile
FROM python:3.11-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application
COPY . .

# Non-root user for security
RUN useradd -m -u 1000 appuser && chown -R appuser:appuser /app
USER appuser

# Expose port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD python -c "import requests; requests.get('http://localhost:8000/health'

# Run application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers",
```

docker-compose.yml:

```yaml
```

```yaml
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://postgres:password@db:5432/investigate
      - REDIS_URL=redis://redis:6379
      - PINECONE_API_KEY=${PINECONE_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
    depends_on:
      - db
      - redis
    restart: unless-stopped

  db:
    image: postgres:15
    environment:
      - POSTGRES_DB=investigate
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    restart: unless-stopped
```

```yaml
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./ssl:/etc/nginx/ssl:ro
    depends_on:
      - api
    restart: unless-stopped

volumes:
  postgres_data:
```

AWS Deployment Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                      CLOUDFLARE CDN                          │
│                  (DDoS Protection, WAF)                      │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│                    AWS ROUTE 53 (DNS)                        │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│          APPLICATION LOAD BALANCER (ALB)                     │
│              (SSL Termination, Health Checks)                │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
```

```
│                 ECS FARGATE CLUSTER                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │
│  │  API Task 1  │  │  API Task 2  │  │  API Task 3  │      │
│  │  (4 vCPU)    │  │  (4 vCPU)    │  │  (4 vCPU)    │      │
│  │  (8 GB RAM)  │  │  (8 GB RAM)  │  │  (8 GB RAM)  │      │
│  └──────────────┘  └──────────────┘  └──────────────┘      │
│       ↓                  ↓                  ↓              │
│  Auto-scaling: 3-10 tasks based on CPU/memory               │
└──────────────────────────────────────────────────────────────┘
                           ↓
┌──────────────────────────────────────────────────────────────┐
│                    RDS POSTGRESQL                            │
│           (db.r6g.xlarge, Multi-AZ)                          │
│              (Read Replicas for scaling)                     │
└──────────────────────────────────────────────────────────────┘
```

CI/CD Pipeline (GitHub Actions)

```yaml
```

```yaml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest pytest-asyncio pytest-cov
      - name: Run tests
        run: pytest --cov=. --cov-report=xml
      - name: Upload coverage
        uses: codecov/codecov-action@v3

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
```

```yaml
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1
      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1
      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          ECR_REPOSITORY: investigate-api
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
          docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to ECS
        run: |
          aws ecs update-service \
            --cluster investigate-prod \
            --service investigate-api \
            --force-new-deployment
```

PERFORMANCE OPTIMIZATION

Caching Strategy

Redis for:

- SEC 10-K filings (24-hour TTL)
- Market data (5-minute TTL)
- User portfolio data (1-hour TTL)

```python
```

```python
import redis.asyncio as redis
from functools import wraps

redis_client = redis.from_url(os.getenv("REDIS_URL"))

def cache(ttl: int = 3600):
    """Caching decorator"""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            # Generate cache key
            cache_key = f"{func.__name__}:{hash(str(args) + str(kwargs))}"

            # Check cache
            cached = await redis_client.get(cache_key)
            if cached:
                return json.loads(cached)

            # Execute function
            result = await func(*args, **kwargs)

            # Store in cache
            await redis_client.setex(
                cache_key,
                ttl,
                json.dumps(result, default=str)
            )

            return result
        return wrapper
    return decorator
```

```python
# Usage
@cache(ttl=86400)  # 24 hours
async def get_sec_filing(ticker: str):
    """Cached SEC filing retrieval"""
    # ... expensive SEC API call
    pass
```

Prompt Caching

Claude Prompt Caching:

```python
```

```python
async def call_llm_with_cache(
    system_prompt: str,
    user_prompt: str,
    cache_system: bool = True
) -> str:
    """Call Claude with prompt caching for system prompts"""

    messages = [
        {
            "role": "system",
            "content": system_prompt,
            "cache_control": {"type": "ephemeral"} if cache_system else None
        },
        {
            "role": "user",
            "content": user_prompt
        }
    ]

    response = await anthropic.messages.create(
        model="claude-3-5-sonnet-20241022",
        max_tokens=2000,
        messages=messages
    )

    return response.content[0].text
```

Cost savings:

- System prompts cached for 5 minutes
- 90% cost reduction on repeated calls with same system prompt

- Example: Bull agent system prompt is identical across analyses

Database Optimization

Indexes:

```sql
-- Speed up user analysis lookups
CREATE INDEX idx_user_analyses ON analyses(user_id, created_at DESC);

-- Speed up ticker searches
CREATE INDEX idx_ticker_analyses ON analyses(ticker, created_at DESC);

-- Speed up portfolio lookups
CREATE INDEX idx_portfolio_user ON portfolios(user_id);

-- JSONB indexes for evidence citations
CREATE INDEX idx_bull_output ON analyses USING gin(bull_output);
CREATE INDEX idx_bear_output ON analyses USING gin(bear_output);
```

Connection Pooling:

```python
```

```python
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker

# Create engine with connection pool
engine = create_async_engine(
    os.getenv("DATABASE_URL"),
    pool_size=20,
    max_overflow=10,
    pool_pre_ping=True,
    echo=False
)

# Session factory
async_session = sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False
)

# Usage
async def get_user(user_id: str):
    async with async_session() as session:
        result = await session.execute(
            select(User).where(User.user_id == user_id)
        )
        return result.scalar_one_or_none()
```

TESTING STRATEGY

Unit Tests

```python
```

```python
import pytest
from unittest.mock import Mock, patch

@pytest.mark.asyncio
async def test_bull_agent_output_schema():
    """Test Bull agent returns valid schema"""

    mock_llm = Mock()
    mock_llm.ainvoke.return_value = Mock(
        content=json.dumps({
            "conviction": 90,
            "best_target": 350,
            "timeline": "3 years",
            "advantages": ["Strong moat"],
            "catalysts": ["AI growth"],
            "valuation": "Justified by growth",
            "evidence_citations": []
        })
    )

    with patch('agents.llm', mock_llm):
        result = await bull_agent(
            ticker="NVDA",
            sec_context="...",
            market_data={},
            rag_context=""
        )

    assert isinstance(result, BullAnalysis)
    assert result.conviction == 90
    assert result.best_target == 350
```

```python
@pytest.mark.asyncio
async def test_evidence_scoring():
    """Test evidence scoring calculation"""

    bull = BullAnalysis(
        conviction=90,
        best_target=350,
        timeline="3 years",
        advantages=[],
        catalysts=[],
        valuation="",
        evidence_citations=[
            Citation(claim="Rev $60B", source="10-K", snippet="...", is_verifie
            Citation(claim="Growth 40%", source="10-K", snippet="...", is_verif
            Citation(claim="Market share 25%", source="estimate", snippet="", i
        ]
    )

    assert bull.verification_rate == pytest.approx(0.666, rel=0.01)

@pytest.mark.asyncio
async def test_hidden_exposure_calculation():
    """Test hidden exposure through ETFs"""

    portfolio = {
        "total_value": 100000,
        "holdings": [
            {"ticker": "SPY", "value": 50000},
            {"ticker": "QQQ", "value": 30000}
        ]
    }
```

```python
    hidden = calculate_hidden_exposure(portfolio, "NVDA")

    # SPY: 50000 * 0.053 = 2650
    # QQQ: 30000 * 0.078 = 2340
    # Total: 4990

    assert len(hidden) == 2
    assert sum(h.hidden_value for h in hidden) == pytest.approx(4990, rel=0.01)
```

Integration Tests

```python
```

```python
@pytest.mark.asyncio
async def test_full_analysis_flow():
    """Test complete analysis from request to response"""

    async with AsyncClient(app=app, base_url="http://test") as client:
        # Create test user
        user_response = await client.post("/auth/register", json={
            "email": "test@example.com",
            "password": "testpass123"
        })
        token = user_response.json()["access_token"]

        # Submit analysis
        analysis_response = await client.post(
            "/analyze",
            headers={"Authorization": f"Bearer {token}"},
            json={
                "ticker": "NVDA",
                "action": "BUY",
                "amount": 5000,
                "portfolio": {
                    "total_value": 100000,
                    "holdings": [{"ticker": "SPY", "value": 50000}]
                }
            }
        )

        assert analysis_response.status_code == 200
        result = analysis_response.json()

        # Verify structure
        assert "bull_analysis" in result
```

```python
        assert "bear_analysis" in result
        assert "portfolio_analysis" in result
        assert "judge_decision" in result

        # Verify decision quality
        assert result["judge_decision"]["confidence"] > 0
        assert result["judge_decision"]["traffic_light"] in ["green", "yellow",
```

Load Testing

```python
```

```python
# locustfile.py
from locust import HttpUser, task, between

class InvestigateUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        """Login before starting tasks"""
        response = self.client.post("/auth/login", json={
            "email": "test@example.com",
            "password": "testpass123"
        })
        self.token = response.json()["access_token"]

    @task
    def analyze_stock(self):
        """Test stock analysis endpoint"""
        self.client.post(
            "/analyze",
            headers={"Authorization": f"Bearer {self.token}"},
            json={
                "ticker": "NVDA",
                "action": "BUY",
                "amount": 5000,
                "portfolio": {"total_value": 100000, "holdings": []}
            }
        )

# Run: locust -f locustfile.py --host=https://api.investigate.ai
```

Performance Targets:

- p50 latency: <15s
- p95 latency: <25s
- p99 latency: <35s
- Throughput: 100 requests/second

MONITORING & OBSERVABILITY

Logging

```python
```

```python
import structlog
from pythonjsonlogger import jsonlogger

# Configure structured logging
structlog.configure(
    processors=[
        structlog.stdlib.add_log_level,
        structlog.stdlib.add_logger_name,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer()
    ],
    logger_factory=structlog.stdlib.LoggerFactory(),
)

logger = structlog.get_logger()

# Usage
@app.post("/analyze")
async def analyze_endpoint(data: AnalysisRequest):
    logger.info(
        "analysis_started",
        ticker=data.ticker,
        action=data.action,
        amount=data.amount
    )

    try:
        result = await analyze_investment(...)

        logger.info(
            "analysis_completed",
            ticker=data.ticker,
```

```python
            traffic_light=result.traffic_light,
            confidence=result.confidence,
            execution_time_ms=result.execution_time_ms
        )

        return result

    except Exception as e:
        logger.error(
            "analysis_failed",
            ticker=data.ticker,
            error=str(e),
            error_type=type(e).__name__
        )
        raise
```

Metrics (Prometheus)

```python
```

```python
from prometheus_client import Counter, Histogram, Gauge

# Define metrics
analyses_total = Counter(
    'analyses_total',
    'Total number of analyses',
    ['ticker', 'traffic_light']
)

analysis_duration = Histogram(
    'analysis_duration_seconds',
    'Analysis execution time',
    buckets=[5, 10, 15, 20, 30, 45, 60]
)

active_analyses = Gauge(
    'active_analyses',
    'Number of currently running analyses'
)

# Usage
@app.post("/analyze")
async def analyze_endpoint(data: AnalysisRequest):
    active_analyses.inc()

    with analysis_duration.time():
        result = await analyze_investment(...)

    analyses_total.labels(
        ticker=data.ticker,
        traffic_light=result.traffic_light
    ).inc()
```

```
    active_analyses.dec()

    return result
```

Alerting

CloudWatch Alarms:

```yaml
```

```yaml
# cloudwatch-alarms.yaml
Alarms:
  HighErrorRate:
    Type: AWS::CloudWatch::Alarm
    Properties:
      AlarmName: investigate-high-error-rate
      MetricName: 5XXError
      Namespace: AWS/ApplicationELB
      Statistic: Sum
      Period: 300
      EvaluationPeriods: 2
      Threshold: 10
      ComparisonOperator: GreaterThanThreshold
      AlarmActions:
        - !Ref SNSTopic

  HighLatency:
    Type: AWS::CloudWatch::Alarm
    Properties:
      AlarmName: investigate-high-latency
      MetricName: TargetResponseTime
      Namespace: AWS/ApplicationELB
      Statistic: Average
      Period: 300
      EvaluationPeriods: 2
      Threshold: 30
      ComparisonOperator: GreaterThanThreshold
      AlarmActions:
        - !Ref SNSTopic
```

DEVELOPMENT SETUP

Local Development

Requirements:

- Python 3.11+
- PostgreSQL 15+
- Redis 7+
- Docker & Docker Compose

Setup:

```bash
```

```bash
# Clone repository
git clone https://github.com/yourusername/investigate.git
cd investigate

# Create virtual environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt

# Set up environment variables
cp .env.example .env
# Edit .env with your API keys

# Start PostgreSQL and Redis
docker-compose up -d db redis

# Run database migrations
alembic upgrade head

# Start development server
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Environment Variables (.env):

```bash
```

```
# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/investigate

# Redis
REDIS_URL=redis://localhost:6379

# AI APIs
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...

# Vector DB
PINECONE_API_KEY=...
PINECONE_ENVIRONMENT=us-east-1

# Data Sources
ALPHA_VANTAGE_API_KEY=...
NEWS_API_KEY=...

# Security
JWT_SECRET_KEY=your-secret-key-here
ENCRYPTION_KEY=your-encryption-key-here

# Plaid
PLAID_CLIENT_ID=...
PLAID_SECRET=...
PLAID_ENV=sandbox

# Monitoring
SENTRY_DSN=https://...
```

Code Quality

Pre-commit hooks:

```yaml
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/psf/black
    rev: 23.3.0
    hooks:
      - id: black

  - repo: https://github.com/pycqa/isort
    rev: 5.12.0
    hooks:
      - id: isort

  - repo: https://github.com/pycqa/flake8
    rev: 6.0.0
    hooks:
      - id: flake8
        args: [--max-line-length=100]

  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: v1.3.0
    hooks:
      - id: mypy
        additional_dependencies: [types-all]
```

Install:

```bash
```

```
pre-commit install
```

APPENDIX

API Rate Limits

Tier	Analyses/Hour	Analyses/Day	Analyses/Month
Free	3	5	10
Premium ($15/mo)	Unlimited	Unlimited	Unlimited
Enterprise	Custom	Custom	Custom

Error Codes

Code	Description
400	Bad Request - Invalid input
401	Unauthorized - Invalid/missing token
403	Forbidden - Rate limit exceeded
404	Not Found - Resource doesn't exist
429	Too Many Requests - Rate limited
500	Internal Server Error - System error

Code	Description
503	Service Unavailable - Temporary outage

Changelog

v3.0 (March 2026):

- Added 4-agent adversarial framework
- Implemented evidence scoring (0-40 scale)
- Added hidden exposure detection
- SEC 10-K grounding integration
- Portfolio-aware recommendations

v2.0 (January 2026):

- Single-agent analysis with RAG
- Basic portfolio integration
- Plaid connectivity

v1.0 (November 2025):

- Initial MVP release
- ChatGPT-based stock analysis

CONTACT

Technical Support: dev@investigate.ai
API Issues: api-support@investigate.ai

Security: security@investigate.ai

Documentation: https://docs.investigate.ai
Status Page: https://status.investigate.ai

Last Updated: March 15, 2026
Document Version: 3.0